

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«МОГИЛЕВСКИЙ ГОСУДАРСТВЕННЫЙ ПОЛИТЕХНИЧЕСКИЙ КОЛЛЕДЖ»

Специальность

5-04-0612-02

Учебный предмет

Инструментальное
программное обеспечение

УТВЕРЖДАЮ

Заместитель директора
по учебной работе

_____ М.М.Федоськова

ЛАБОРАТОРНАЯ РАБОТА № 22

СОЗДАНИЕ СТРУКТУРЫ ШАБЛОНОВ И ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА ВЕБ-ПРИЛОЖЕНИЯ В DJANGO

МЕТОДИЧЕСКИЕ РЕКОМЕНДАЦИИ

Разработал

преподаватель
Денисовец Д.А.

Обсуждено и одобрено
на заседании цикловой комиссии
специальностей в области
программного обеспечения
информационных систем
Протокол № _____ от _____
Председатель цикловой комиссии
_____ Д.А.Денисовец

1 Цель работы

1.1 Создание структуры шаблонов и пользовательского интерфейса веб-приложения в Django

2 Методическое и материальное обеспечение

2.1 Персональный компьютер

2.2 Методические рекомендации по выполнению лабораторной работы

2.3 Интегрированная среда разработки PyCharm / Visual Studio Code

3 Последовательность выполнения работы

3.1 Ознакомиться с основными теоретическими положениями

3.2 Выполнить индивидуальное задание

3.3 Оформить отчет

3.4 Ответить на контрольные вопросы

4 Основные теоретические положения

4.1 Создание и использование шаблонов

Шаблоны (templates) отвечают за формирование внешнего вида приложения. Они предоставляют специальный синтаксис, который позволяет внедрять данные в код HTML.

Допустим, у нас есть проект metanit, и в нем определено одно приложение – hello (рисунок 1):

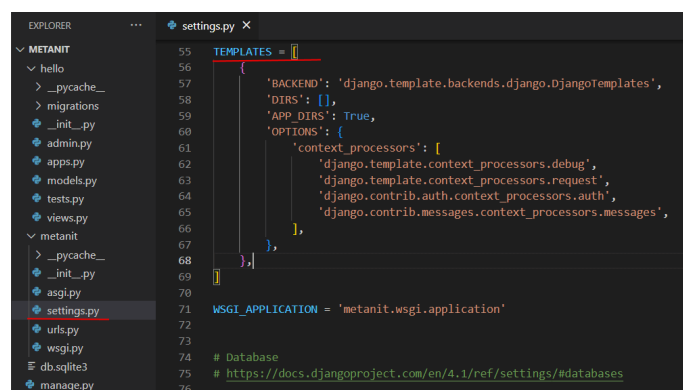


Рисунок 1 - Шаблоны Templates в Django

Настройка функциональности шаблонов в проекте Django производится в файле settings.py. с помощью переменной TEMPLATES. Так, по умолчанию переменная TEMPLATES в файле settings.py имеет следующее определение:

```

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',

```

```

'DIRS': [],
'APP_DIRS': True,
'OPTIONS': {
    'context_processors': [
        'django.template.context_processors.debug',
        'django.template.context_processors.request',
        'django.contrib.auth.context_processors.auth',
        'django.contrib.messages.context_processors.messages',
    ],
},
],
},
]

```

Данная переменная принимает список конфигураций для каждого движка шаблонов. По умолчанию определена одна конфигурация, которая имеет следующие параметры:

- **BACKEND**: движок шаблонов. По умолчанию применяется встроенный движок `django.template.backends.django.DjangoTemplates`;
- **DIRS**: определяет список каталогов, где движок шаблонов будет искать файлы шаблонов. По умолчанию пустой список;
- **APP_DIRS**: указывает, будет ли движок шаблонов искать шаблоны внутри папок приложений в папке `templates`;
- **OPTIONS**: определяет дополнительный список параметров

Итак, в конфигурации по умолчанию параметр `APP_DIRS` имеет значение `True`, а это значит, что движок шаблонов будет также искать нужные файлы шаблонов в папке приложения в каталоге `templates`. То есть по умолчанию мы уже имеем настроенную конфигурацию, готовую к использованию шаблонов. Теперь определим сами шаблоны.

Добавим в папку приложения каталог `templates`. А в нем определим файл `index.html` (рисунок 2):

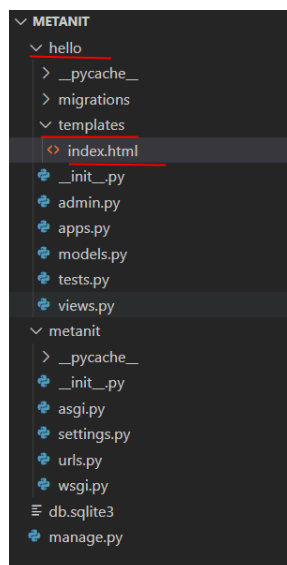


Рисунок 2 - Добавление шаблонов Templates в проект на Django и Python

Далее в файле index.html определим следующий код:

```
<!DOCTYPE html>
<html>
<head>
  <title>Django на METANIT.COM</title>
  <meta charset="utf-8" />
</head>
<body>
  <h2>Hello METANIT.COM</h2>
</body>
</html>
```

По сути, это обычная веб-страница, которая содержит код html. Теперь используем эту страницу для отправки ответа пользователю. И для этого перейдем в приложении hello к файлу views.py, который определяет функции для обработки запроса. Изменим этот файл следующим образом:

```
from django.shortcuts import render
def index(request):
    return render(request, "index.html")
```

Из модуля django.shortcuts импортируется функция render.

Функция index вызывает функцию render, которой передаются объект запроса request и путь к файлу шаблона в рамках папки templates - "index.html".

В файле urls.py проекта пропишем сопоставление функции index с запросом к корню веб-приложения (рисунок 3):

```
from django.urls import path
from hello import views
urlpatterns = [
    path("", views.index),
]
```

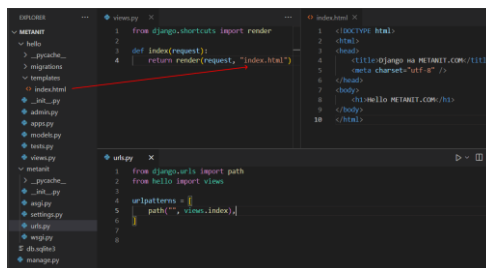


Рисунок 3 - Связь шаблона template и view в Django

И запустим проект на выполнение и перейдем к приложению в браузере (если проект запущен, то его надо перезапустить) (рисунок 4):

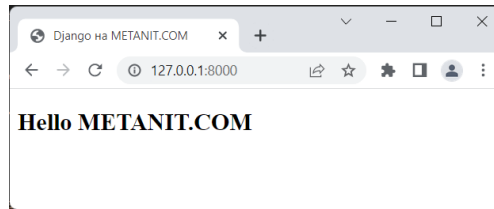


Рисунок 4 - Шаблоны в веб-приложении на Django и python

Подобным образом можно указать и другие шаблоны. Например, в папку templates добавим еще две страницы: about.html и contact.html (рисунок 5).

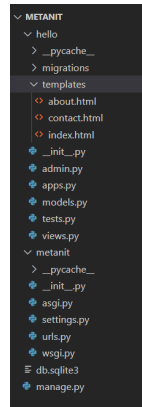


Рисунок 5 - Шаблоны в Python и Django

И также в файле views.py определим функции, которые используют данные шаблоны:

```
from django.shortcuts import render
def index(request):
    return render(request, "index.html")
def about(request):
    return render(request, "about.html")
def contact(request):
    return render(request, "contact.html")
```

А в файле urls.py свяжем функции с маршрутами:

```
from django.urls import path
from hello import views
urlpatterns = [
    path("", views.index),
    path("about/", views.about),
    path("contact/", views.contact),
]
```

Выше для генерации шаблона применялась функция render(), которая является наиболее распространенным вариантом. Однако также мы можем использовать класс TemplateResponse:

```
from django.template.response import TemplateResponse
def index(request):
    return TemplateResponse(request, "index.html")
```

Результат будет тот же самый.

4.2 Обработка запроса

Центральным моментом любого веб-приложения является обработка запроса, который отправляет пользователь. В Django за обработку запроса отвечают представления или `views`. По сути представления представляют функции обработки, которые принимают данные запроса в виде объекта `HttpRequest` из пакета `django.http` и генерируют некоторый результат, который затем отправляется пользователю.

По умолчанию представления размещаются в приложении в файле `views.py`.

Например, возьмем стандартный проект, в который добавлено приложение (рисунок 6).

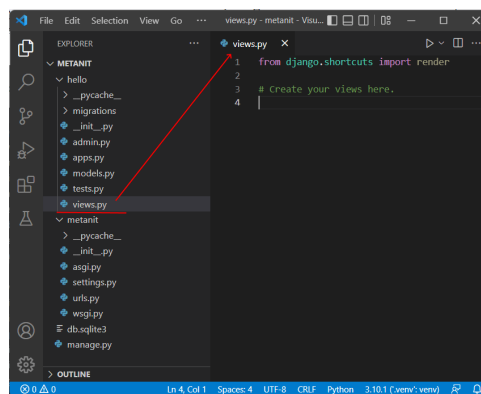


Рисунок 6 - Представления Views в приложении на Django и Python

При создании нового проекта файл `views.py` имеет следующее содержимое:

```
from django.shortcuts import render
# Create your views here.
```

Данный код пока никак не обрабатывает запросы, он только импортирует функцию `render()`, которая может использоваться для обработки.

Генерировать результат можно различными способами. Один из них представляет использование класса `HttpResponse` из пакета `django.http`, который позволяет отправить текстовое содержимое.

Так, изменим файл `views.py` следующим образом:

```
from django.http import HttpResponse
def index(request):
    return HttpResponse("Главная")
def about(request):
    return HttpResponse("О сайте")
def contact(request):
    return HttpResponse("Контакты")
```

В данном случае определены три функции, которые будут обрабатывать запросы. Каждая функция принимает в качестве параметра `request` объект

HttpRequest, который хранит информацию о запросе. Однако в данном случае она нам не нужна, поэтому параметр никак не используется. Для генерации ответа в конструктор объекта HttpResponse передается некоторая строка. Это может быть в том числе и код html в виде строки.

Чтобы эти функции сопоставлялись с запросами, надо определить для них маршруты в проекте в файле urls.py. В частности, изменим этот файл следующим образом (рисунок 7):

```
from django.urls import path
from hello import views
urlpatterns = [
    path("", views.index),
    path('about', views.about),
    path('contact', views.contact),
]
```

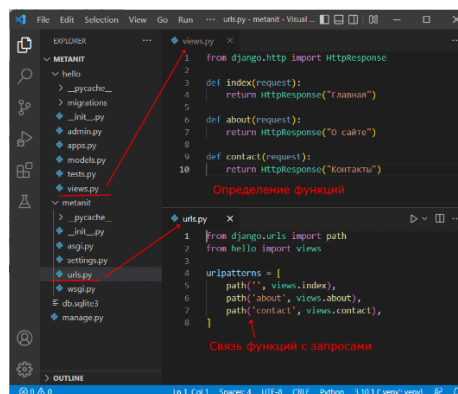


Рисунок 7 - Обработка запроса в Django и Python

Переменная `urlpatterns` определяет набор сопоставлений функций обработки с определенными строками запроса. Например, запрос к корню веб-сайта будет обрабатываться функцией `index`, запрос по адресу "about" будет обрабатываться функцией `about`, а запрос "contact" - функцией `contact`.

Запустим проект и обратимся по некоторым из этих адресов (рисунок 8).

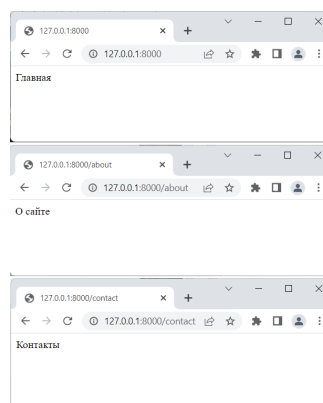


Рисунок 8 - Сопоставление запросов с функциями в Django и Python

При этом мы можем отправлять не простой текст, а, например, код html, который затем интерпретируется браузером. Так, изменим файл `views.py` следующим образом:

```
from django.http import HttpResponse
def index(request):
    return HttpResponse("<h2>Главная</h2>")
def about(request):
    return HttpResponse("<h2>О сайте</h2>")
def contact(request):
    return HttpResponse("<h2>Контакты</h2>")
```

Соответственно теперь браузер получит код html (рисунок 9):

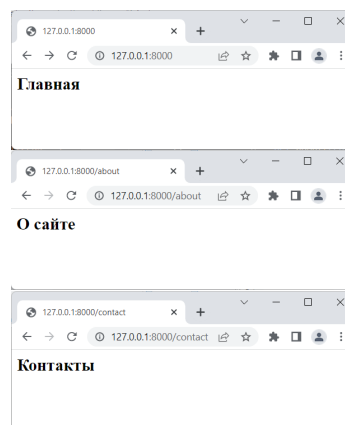


Рисунок 9 - Обработка запросов и `HttpResponse` в Django и Python

4.3 Практическая часть

Центральным моментом любого веб-приложения является обработка запроса, который отправляет пользователь. В Django за обработку запроса отвечают представления или `views`. По сути представления представляют функции обработки, которые принимают данные запроса в виде объекта `HttpRequest` из пакета `django.http` и генерируют некоторый результат, который затем отправляется пользователю.

Обработка запроса — процесс принятия и обработки данных запроса пользователя в веб-приложении.

По умолчанию представления размещаются в приложении в файле `views.py`. Например, возьмем стандартный проект, в который добавлено приложение (рисунок 10).

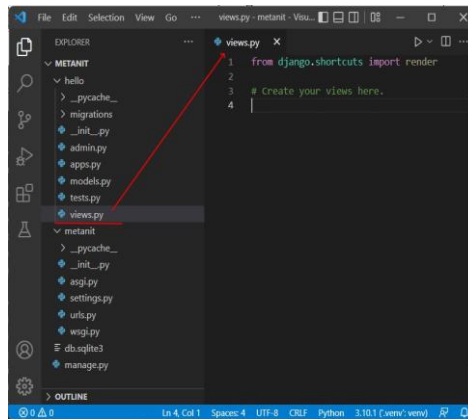


Рисунок 10 - Файл views.py

При создании нового проекта файл **views.py** имеет следующее содержимое (рисунок 11):

```
1  from django.shortcuts import render
2
3  # Create your views here.
```

Рисунок 11 - Базовый вид файла views.py

Данный код пока никак не обрабатывает запросы, он только импортирует функцию **render()**, которая может использоваться для обработки.

Генерировать результат можно различными способами. Один из них представляет использование класса **HttpResponse** из пакета **django.http**, который позволяет отправить текстовое содержимое.

Так, изменим файл **views.py** следующим образом (рисунок 12):

```
views.py > ...
1  from django.http import HttpResponse
2
3  def index(request):
4      return HttpResponse("Главная")
5
6  def about(request):
7      return HttpResponse("О сайте")
8
9  def contact(request):
10     return HttpResponse("Контакты")
```

Рисунок 12 - Измененный файл с функциями для отображения главной страницы, страницы о сайте и контактов

В данном случае определены три функции, которые будут обрабатывать запросы. Каждая функция принимает в качестве параметра **request** объект **HttpRequest**, который хранит информацию о запросе. Однако в данном случае она нам не нужны, поэтому параметр никак не используется. Для генерации ответа в конструктор объекта **HttpResponse** передается некоторая строка. Это может быть в том числе и код **html** в виде строки.

Чтобы эти функции сопоставлялись с запросами, надо определить для них маршруты в проекте в файле **urls.py**. В частности, изменим этот файл следующим образом (рисунок 13):

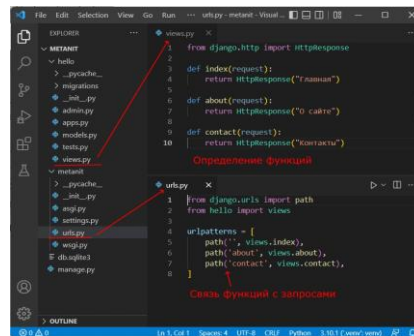


Рисунок 13 - Связь между файлами views.py и urls.py

Переменная **urlpatterns** определяет набор сопоставлений функций обработки с определенными строками запроса. Например, запрос к корню веб-сайта будет обрабатываться функцией **index**, запрос по адресу "about" будет обрабатываться функцией **about**, а запрос "contact" - функцией **contact**.

Запустим проект и обратимся по некоторым из этих адресов (рисунок 14).



Рисунок 14 - Отображение наших представлений в браузере

Вернемся к нашему проекту и перейдем в файл **views.py**. У уже известной функции **render** – есть такой параметр, как **context**, он и позволяет нам выводить определенные данные на сайте (если логика прописана правильно) (рисунок 15).

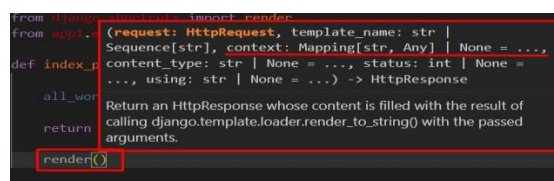
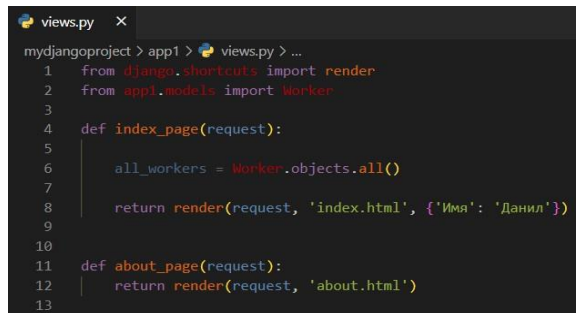


Рисунок 15 - Подсказка в Visual Studio Code о параметрах функций

Передадим в данный context словарь (пара «ключ-значение») какие-нибудь данные, например (рисунок 16):



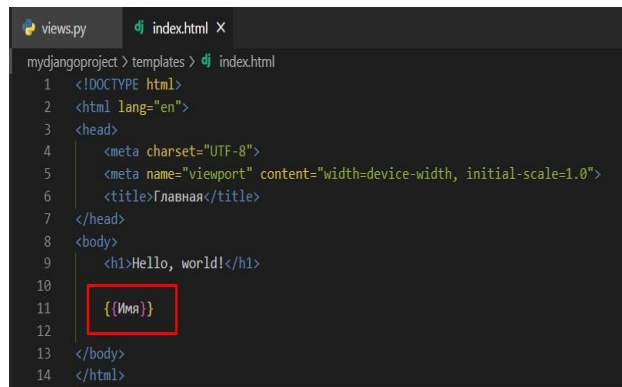
```

mydjango> app1 > views.py > ...
1  from django.shortcuts import render
2  from app1.models import Worker
3
4  def index_page(request):
5
6      all_workers = Worker.objects.all()
7
8      return render(request, 'index.html', {'Имя': 'Данил'})
9
10
11 def about_page(request):
12     return render(request, 'about.html')
13

```

Рисунок 16 - Передача данных в context

Перейдем в index.html и укажем в двойных фигурных скобках значение КЛЮЧА словаря (рисунок 17).



```

mydjango> templates > index.html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Главная</title>
7  </head>
8  <body>
9      <h1>Hello, world!</h1>
10     {{Имя}}
11
12 </body>
13 </html>

```

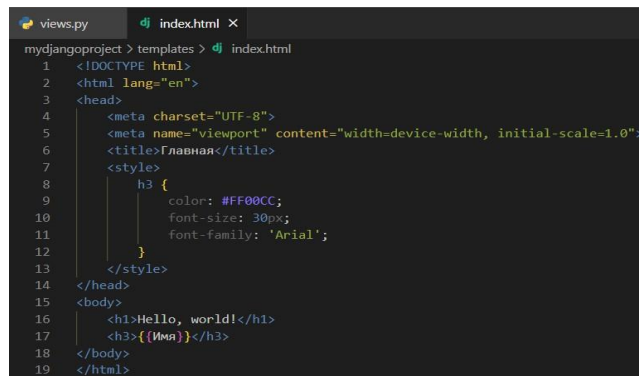
Рисунок 17 - Код, для того чтобы отобразить данные на странице

Обновим страницу. И что получим (рисунок 18)?



Рисунок 18 - Результат отображения на странице

С переданным аргументом можно делать все что угодно, например, изменять стили. Изменим код для тега head, добавив стили. И изменим немного шрифт, добавим данный текст в заголовок третьего уровня и, к примеру, добавим подчеркивание для текста (рисунок 19).



```

mydjango project > templates > dj index.html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Главная</title>
7   <style>
8     h3 {
9       color: #FF00CC;
10      font-size: 30px;
11      font-family: 'Arial';
12    }
13  </style>
14 </head>
15 <body>
16   <h1>Hello, world!</h1>
17   <h3>{{Имя}}</h3>
18 </body>
19 </html>

```

Рисунок 19 - Добавление стилей

Проверим, как все отображается на странице и убедимся, что все работает (рисунок 20).

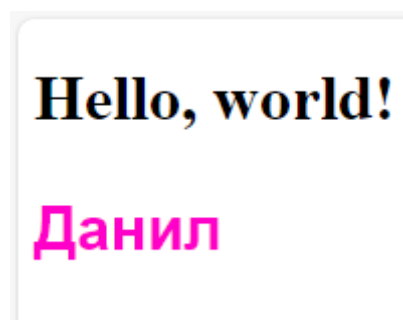
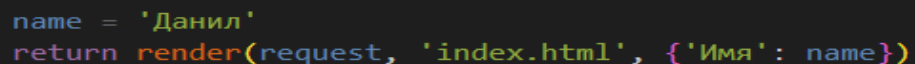


Рисунок 20 - Корректная работа стилей

Используя знания питона, несложно догадаться, что значения в словарь, можно передавать ещё и через переменную. Например (рисунок 21):



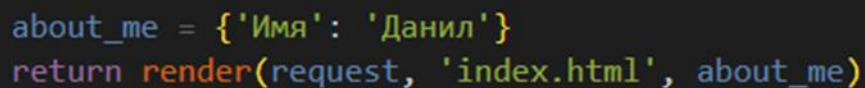
```

name = 'Данил'
return render(request, 'index.html', {'Имя': name})

```

Рисунок 21 - Передача значений в контексте в виде переменной

Или передать сразу словарь в виде переменной (рисунок 22):



```

about_me = {'Имя': 'Данил'}
return render(request, 'index.html', about_me)

```

Рисунок 22 - Передача словаря в виде контекста

Принцип думаю понятен. Можно передавать и списки, и вложенные словари. Создадим новый ключ `workers` и в качестве значения передадим список всех работников (вспоминаем предыдущую практическую работу). А также, в файле `index.html` поменяем название ключа (рисунок 23).

```

mydjango> project > app1 > views.py > ...
1  from django.shortcuts import render
2  from app1.models import Worker
3
4  def index_page(request):
5
6      all_workers = Worker.objects.all()
7
8      return render(request, 'index.html', {'workers': all_workers})
9
10
11 def about_page(request):
12     return render(request, 'about.html')
13

```

Рисунок 23 - Изменение названия ключа для списка всех сотрудников на workers

Но, что мы видим (рисунок 24)?

Hello, world!

<QuerySet [<Worker: Газизуллин Данил>, <Worker: Иванов Иван>, <Worker: Дементьев Алексей>, <Worker: Петров Арсений>, <Worker: Захаров Михаил>]>

Рисунок 24 - Вывод данных на странице

В таком виде, нам нельзя выводить информацию пользователю. Для того, чтобы нормализовать вывод необходимо использовать язык шаблонов Django, или Django Template Language (Далее – DTL)

Django Template Language — текстовый язык шаблонов, который обеспечивает мост между сценариями (HTML, CSS, JS и т. д.) и языками программирования (Python)

По сути, шаблон Django – обычный html-файл, но с большими возможностями.

Что из этого следует понять? Стоит ли писать на языке шаблонов Django, как на отдельном языке программирования, нет, ни в коем случае! Можно реализовать циклы, условия и прочее с помощью данного языка. Засовывать в текст HTML какие-то немудрёные расчеты и программную логику – не нужно! По-хорошему все делается в бекенде и во фронтенд передавать уже готовые данные, но если прямо присутствует необходимость сделать логику во фронтенд-части, то используют JavaScript.

Реализуем цикл for для того, чтобы вывести каждого работника по отдельности (рисунок 25). Результат представлен на рисунке 26.

```
<body>
  <h1>Hello, world!</h1>

  {% for i in workers %}
    {{ i }}
  {% endfor %}
</body>
```

Рисунок 25 - Цикл for на языке DTL

Hello, world!

Газизуллин Данил Иванов Иван Дементьев Алексей Петров Арсений Захаров Михаил

Рисунок 26 - Отображение сотрудников

Отлично, но что делать, если все вывелось в одну строку и у нас нет оформления стилей для заголовка третьего уровня. Все просто, обернем нашу итерируемый объект в тег заголовка третьего уровня (рисунок 27).

```
<body>
  <h1>Hello, world!</h1>

  {% for i in workers %}
    <h3>{{ i }}</h3>
  {% endfor %}
</body>
```

Рисунок 27 - Корректировка цикла для отображения

И получим красивый результат (рисунок 28):

Hello, world!

Газизуллин Данил

Иванов Иван

Дементьев Алексей

Петров Арсений

Захаров Михаил

Рисунок 28 - Красивый вывод сотрудников

В DTL есть некая «внутренняя переменная», которая называется `forloop.counter`. Она считает количество итераций цикла (похожая функция в Python – `enumerate`). Добавим вывод № сотрудника перед каждой итерацией в заголовок второго уровня (рисунок 29).

```
<body>
  <h1>Список сотрудников</h1>

  {% for i in workers %}
    <h2>Сотрудник №{{ forloop.counter }}</h2>
    <h3>{{ i }}</h3>
  {% endfor %}
</body>
```

Рисунок 29 - Добавление счетчика итераций в цикл for

Приведем все в красивый вид и получим (рисунок 30):

```
<style>
  h2, h3 {
    font-family: 'Arial';
    font-size: 14px;
  }
  h2 {
    color: #4682B4;
  }
  h3 {
    color: #CD5C5C;
  }
</style>
```

Список сотрудников

Сотрудник №1

Газизуллин Данил

Сотрудник №2

Иванов Иван

Сотрудник №3

Дементьев Алексей

Сотрудник №4

Петров Арсений

Сотрудник №5

Захаров Михаил

Рисунок 30 - Корректировка стилей для отображения и результат

Если посмотреть исходный код страницы, все выглядит как обычные HTML-теги (рисунок 31).

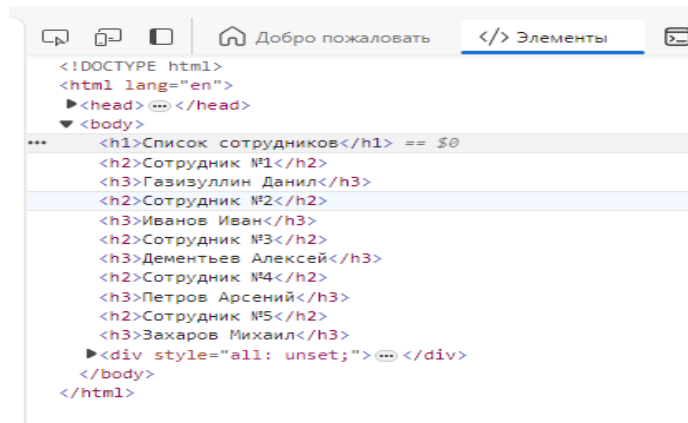


Рисунок 31 - Исходный код страницы

А что будет, если мы в `models.py` удалим строчку кода, которая отвечает за определение строкового представления? (`_str_`). В данном случае, закомментируем её (рисунок 32).

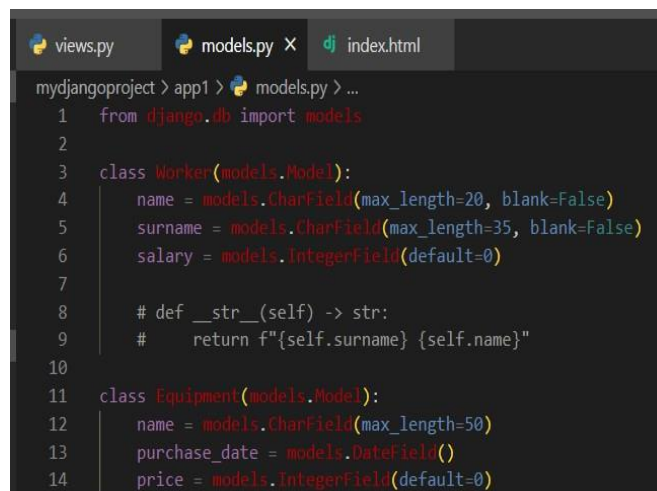


Рисунок 32 - Комментирование строкового представления для модели Worker

Если обновим страницу, увидим следующий беспорядок (рисунок 33):



Рисунок 33 - Цикл с плохим отображением сотрудников

Поэтому, важно научиться обращаться конкретно напрямую к значению. Делается все точно также, как мы учились делать это через обращение к значению класса (рисунок 34).

```
<h1>Список сотрудников</h1>

{% for i in workers %}
    <h2>Сотрудник №{{ forloop.counter }}</h2>
    <h3>{{ i.surname }} {{ i.name }} {{ i.salary }}</h3>
{% endfor %}
```

Рисунок 34 - Редактирование цикла for

Также, забегаая вперед, отмечу, что точка, используется ещё для обращения по индексу. Если в питоне мы используем для этого квадратные скобки, то в DTL необходимо писать через точку (рисунок 35).

```
users = ['Данил', 'Иван', 'Михаил']
print(users[0]) # Выведет Данил
```

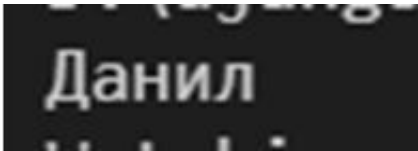


Рисунок 35 - Обращение по индексу в списках в Python

Эквивалент на DTL (рисунок 36):

```
def index_page(request):

    all_workers = Worker.objects.all()
    users = ['Данил', 'Иван', 'Михаил']
    return render(request, 'index.html', {'users': users})
```

Рисунок 36 - Передача списка пользователей в контекст

В файле HTML пропишем название переданной переменной пользователей (users), а также обратимся по индексу к именам (рисунок 37).

<pre><h1>Эквивалент</h1> {{ users }}
 {{ users.0 }}
 {{ users.1 }}
 {{ users.2 }}</pre>	Эквивалент <pre>['Данил', 'Иван', 'Михаил'] Данил Иван Михаил</pre>
---	---

Рисунок 37 - Отображение списка пользователей, а также обращение к ним по индексу

Условно, давайте выделим зарплату сотрудников зеленым цветом, если она больше 50 000 рублей, если меньше, то красный.

Условие **if** в языке шаблонов Django позволяет условно отображать части шаблона на основе значений переменных, выражений или сравнений (рисунок 38). Результат представлен на рисунке 39.

```
{% for i in workers %}
<h2>Сотрудник №{{ forloop.counter }}</h2>
<h3>
    Фамилия: {{ i.surname }} <br>
    Имя: {{ i.name }} <br>
    Зарплата: {% if i.salary >= 50000 %}
        <b style='color: green'>{{ i.salary }}</b><br>
    {% endif %}
</h3>
{% endfor %}
```

Рисунок 38 - Условие, при котором те сотрудники, у которых salary > 50000 – будут отображаться зелеными



Рисунок 39 - Отображение списка сотрудников с зарплатой выше 50000

На данном этапе мы видим, что если мы не добавим условие **else**, то зарплата тех сотрудников, которая меньше 50 000 рублей, не выведется (рисунок 40). Добавим блок **else**:

```
<h1>Список сотрудников</h1>

{% for i in workers %}
  <h2>Сотрудник №{{ forloop.counter }}</h2>
  <h3>
    Фамилия: {{ i.surname }} <br>
    Имя: {{ i.name }} <br>
    Зарплата: {% if i.salary >= 50000 %}
      <b style='color: green'>{{ i.salary }}</b><br>
    {% else %}
      <b style='color: red'>{{ i.salary }}</b><br>
    {% endif %}
  </h3>
{% endfor %}
```

Рисунок 40 - Добавление блока **else**

В конечном итоге, мы получим такой результат (рисунок 41):



Рисунок 41 - Корректное отображение списка сотрудников

4.5 Пример решения задачи на языке программирования Python

Задача. В ранее созданном проекте `blog_site` и приложении `posts` создайте папку `templates/posts`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `post_list.html`, который наследуется от `base.html` и выводит список публикаций блога из базы данных. Реализуйте представление `post_list` в файле `views.py`, передающее данные модели `Post` в шаблон. Настройте маршрут в файле `urls.py` для отображения списка публикаций и убедитесь, что страница корректно отображается в браузере.

Решение задачи:

1 Представление (`views.py`)

```
from django.shortcuts import render
from .models import Post
def post_list(request):
    posts = Post.objects.all()
    return render(request, 'posts/post_list.html', {'posts': posts})
```

2 Маршруты приложения (`posts/urls.py`)

```
from django.urls import path
from . import views
urlpatterns = [
    path("", views.post_list, name='post_list'),
]
```

3 Подключение маршрутов приложения (`blog_site/urls.py`)

```
from django.contrib import admin
from django.urls import path, include
```

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path("", include('posts.urls')),
]
```

4 Базовый шаблон (templates/posts/base.html)

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="UTF-8">
    <title>Блог</title>
</head>
<body>
    <h1>Блог публикаций</h1>
    <nav>
        <a href="/">Главная</a>
    </nav>
    <hr>
    {% block content %}
    {% endblock %}
</body>
</html>
```

5 Шаблон списка публикаций (templates/posts/post_list.html)

```
{% extends 'posts/base.html' %}
{% block content %}
<h2>Список публикаций</h2>
<ul>
    {% for post in posts %}
        <li>
            <strong>{{ post.title }}</strong><br>
            Автор: {{ post.author }}<br>
            Дата: {{ post.created_at }}
        </li>
        <br>
    {% empty %}
        <li>Публикации отсутствуют.</li>
    {% endfor %}
</ul>
```

```
{% endblock %}
```

6 Проверка работы

Запустите сервер:

```
python manage.py runserver
```

Откройте в браузере:

```
http://127.0.0.1:8000/
```

5 Индивидуальное задание

5.1 Напишите программу решения задачи на языке программирования Python в соответствии с заданным вариантом. Выполните отладку и результат вычислений выведите на экран.

1 В ранее созданном проекте `library_site` и приложении `books` создайте папку `templates/books`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `book_list.html`, который наследуется от `base.html` и выводит список книг из базы данных. Реализуйте представление `book_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

2 В ранее созданном проекте `student_portal` и приложении `students` создайте папку `templates/students`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `student_list.html`, который наследуется от `base.html` и выводит список студентов из базы данных. Реализуйте представление `student_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

3 В ранее созданном проекте `movie_portal` и приложении `films` создайте папку `templates/films`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `film_list.html`, который наследуется от `base.html` и выводит список фильмов из базы данных. Реализуйте представление `film_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

4 В ранее созданном проекте `music_library` и приложении `albums` создайте папку `templates/albums`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `album_list.html`, который наследуется от `base.html` и выводит список музыкальных альбомов из базы данных. Реализуйте представление `album_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

5 В ранее созданном проекте `online_shop` и приложении `products` создайте папку `templates/products`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `product_list.html`, который наследуется от `base.html` и выводит список товаров интернет-магазина из базы данных. Реализуйте представление `product_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

6 В ранее созданном проекте `news_portal` и приложении `news` создайте папку `templates/news`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `news_list.html`, который наследуется от `base.html` и выводит список новостей из базы данных. Реализуйте представление `news_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

7 В ранее созданном проекте `tourism_portal` и приложении `tours` создайте папку `templates/tours`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`.

Затем создайте шаблон `tour_list.html`, который наследуется от `base.html` и выводит список туристических маршрутов из базы данных. Реализуйте представление `tour_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

8 В ранее созданном проекте `sport_club_site` и приложении `teams` создайте папку `templates/teams`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `team_list.html`, который наследуется от `base.html` и выводит список спортивных команд из базы данных. Реализуйте представление `team_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

9 В ранее созданном проекте `school_portal` и приложении `teachers` создайте папку `templates/teachers`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `teacher_list.html`, который наследуется от `base.html` и выводит список преподавателей из базы данных. Реализуйте представление `teacher_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

10 В ранее созданном проекте `hospital_system` и приложении `patients` создайте папку `templates/patients`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `patient_list.html`, который наследуется от `base.html` и выводит список пациентов из базы данных. Реализуйте представление `patient_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

11 В ранее созданном проекте `education_platform` и приложении `courses` создайте папку `templates/courses`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `course_list.html`, который наследуется от `base.html` и выводит список учебных курсов из базы данных. Реализуйте представление `course_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

12 В ранее созданном проекте `restaurant_site` и приложении `menu` создайте папку `templates/menu`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `dish_list.html`, который наследуется от `base.html` и выводит список блюд ресторана из базы данных. Реализуйте представление `dish_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

13 В ранее созданном проекте `car_catalog` и приложении `cars` создайте папку `templates/cars`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `car_list.html`, который наследуется от `base.html` и выводит список автомобилей из базы данных. Реализуйте представление `car_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

14 В ранее созданном проекте `portfolio_site` и приложении `main` создайте папку `templates/main`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `project_list.html`, который наследуется от `base.html` и выводит список проектов портфолио из базы данных. Реализуйте представление `project_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

15 В ранее созданном проекте `home_site` и приложении `home` создайте папку `templates/home`. В этой папке создайте шаблон `base.html`, содержащий базовую структуру сайта: заголовок страницы, меню навигации и блок `content`. Затем создайте шаблон `page_list.html`, который наследуется от `base.html` и выводит список страниц сайта из базы данных. Реализуйте представление `page_list` в файле `views.py` и настройте маршрут в файле `urls.py`.

6 Содержание отчета (отчет по лабораторной работе выполняется в электронном виде в папке учащегося)

- 6.1 Тема лабораторной работы
- 6.2 Цель лабораторной работы
- 6.3 Индивидуальное задание (тексты программ и скриншоты выполнения)

7 Контрольные вопросы

- 7.1 Что такое шаблоны в Django в языке программирования Python?
- 7.2 Что такое наследование шаблонов в языке программирования Python?
- 7.3 Как реализуется цикл в шаблоне Django в языке программирования Python?
- 7.4 Что такое пользовательский интерфейс веб-приложения в языке программирования Python?
- 7.5 Как подключить CSS-файл в Django-шаблоне в языке программирования Python?
- 7.6 Как использовать условные операторы в шаблонах в языке программирования Python?

Список используемых источников

- 1 Постолиит, А. В. Python, Django и PyCharm для начинающих / А. В. Постолиит. – СПб.: БХВ-Петербург, 2021. – 464 с.: ил.
- 2 Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2019. – 608 с.: ил.
- 3 Фоминых, Е. И. Инструментальное программное обеспечение: учебное пособие / Е. И. Фоминых, Т. Е. Фоминых. – Минск : РИПО, 2022. – 410 с.: ил.